■ ALBEREI · DOCUMENT 01

Integration Guide.

Everything an engineer needs to ship Alberei into a product. Read this first.

AUDIENCE	VERSION	ISSUED
Engineers, integrators	1.0	May 2026

01 · THE SHAPE OF THE INTEGRATION

One endpoint, one token.

Alberei is a REST surface over HTTPS. There is one core endpoint: `POST /alberei/v1/generate`. You send a context, a language, and a register. You get back items. There is no SDK to install, no persistent connection to maintain, no client state to manage. A correct integration is twenty lines of code.

The full reference, including all status codes and rate-limit headers, is at `alberei.de/api`. This guide covers what that reference does not: the decisions you make once and never revisit.

02 · GETTING A TOKEN

Access is gated.

Tokens are issued through an invite flow. The fastest path is the access request form at `alberei.de/api#signup` or an email to `api@alberei.de`. Include:

- Your organisation and contact
- The product Alberei will live inside, in one sentence
- Your expected monthly call volume
- The tier you have in mind

Turnaround is typically under two working days. The token arrives by email. Treat it as a secret: store it in your secret manager, inject it as an environment variable, never commit it.

> **Token format.** Tokens are opaque strings prefixed with `alb_tok_`. Length is stable. They are scoped to a single tier; if you upgrade, a new token is issued and the old one continues to work for thirty days.

03 · THE SMALLEST CORRECT CALL

JavaScript, Python, curl.

JavaScript (Node 18 and later, or browser with the right CORS origin)

```
const r = await fetch('https://api.llulllab.com/alberei/v1/generate', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'X-Api-Token': process.env.ALBEREI_TOKEN
  },
  body: JSON.stringify({
    lang: 'en',
    context: 'a robot apologising for being late',
    count: 3,
    register: 'warm'
  })
});
if (!r.ok) throw new Error(`Alberei ${r.status}`);
const { items } = await r.json();
const choice = items[Math.floor(Math.random() * items.length)];
console.log(choice.text, '(', choice.structure, ')');
```

Python (requests)

```
import os, requests
r = requests.post(
  'https://api.llulllab.com/alberei/v1/generate',
  headers={'X-Api-Token': os.environ['ALBEREI_TOKEN'],
           'Content-Type': 'application/json'},
  json={'lang': 'en', 'context': context,
        'count': 3, 'register': 'wry'},
  timeout=10
)
r.raise_for_status()
items = r.json()['items']
```

curl

```
curl -X POST https://api.llulllab.com/alberei/v1/generate \
  -H "Content-Type: application/json" \
  -H "X-Api-Token: $ALBEREI_TOKEN" \
  -d '{"lang":"en","context":"...","count":3,"register":"wry"}'
```

04 · DECISIONS TO MAKE ONCE

Configure, then forget.

Pick a register and stay there

Register is a product decision, not a per-call lever. A wellbeing companion stays in `warm`. A productivity assistant with attitude stays in `wry`. Switching register inside a session breaks the user's sense of persona. Pick one. Document it. Move on.

Pick a length

Spoken delivery: `short`. Written delivery with screen real estate: `medium`. Both are caps, not targets; you may get something shorter.

Decide how many items to ask for

If you display all items (a writing tool, a designer's mood board): `count: 3` to `5`. If you pick one server-side and discard the rest: `count: 2`. Asking for more than you use wastes quota.

Decide when to call

Not every turn. Humour that fires constantly stops being humour. Gate on a context classifier, a user signal, or a long silence. The expensive call is the one that lands in the wrong moment.

05 · ERROR HANDLING

Plan for refusal and quota.

Three errors deserve explicit handling. Treat the rest as transient and retry once with jitter.

STATUS	ERROR	WHAT TO DO
402	`quota_exceeded`	Fall back silently. Check `Retry-After` for backoff. Alert your ops channel if it persists past the period reset.
422	`unsafe_context`	Skip humour for this turn. Do not surface a "rejected" message to the user.
429	`rate_limited`	Honour `Retry-After`. Apply a token bucket on your side to stay under burst.
5xx	`upstream_unreachable` etc.	One retry with 250 ms jitter. After that, silent fallback.

06 · CACHING

When repetition kills, when it does not.

User-facing utterances should not be cached per user. If the user hears the same line twice, the humour is dead.

Abstract beats may be cached freely. A game quest's "first failure" beat, a marketing copy slot, a recurring NPC line. The cache key should include `lang`, `register`, and a deterministic hash of the abstract context. Refresh when content updates.

> **A reasonable budget.** A game pre-generates ten items per NPC per beat per language at content-update time. The runtime samples without calling Alberei in the hot path. The total quota cost is bounded by content surface, not by playtime.

07 · QUOTA AND OBSERVABILITY

Watch the headers.

Every successful response includes `X-RateLimit-Daily-Remaining`, `X-RateLimit-Monthly-Remaining`, and `X-RateLimit-Burst-Remaining`. Read them, log them, alert when monthly remaining crosses below twenty percent of quota.

`GET /alberei/v1/usage` returns the current state without consuming a call. Use it on a dashboard.

08 · WHERE THIS CAN GO WRONG

The common pitfalls.

- **Calling on every turn.** Quota burns fast and humour gets cheap. Gate.
- **Asking for `count: 5` when you display one.** You paid for five. Use them or ask for fewer.
- **Showing the rejection.** A `422` means skip, not "I refused that for you."
- **Mixing registers in a session.** The user notices, even if they cannot name it.
- **Caching per user.** The first laugh costs trust the second time.
- **Hard failure on 5xx.** Silence is a valid response from a humour module.

09 · WHERE TO GO FROM HERE

Companion documents.

METHODOLOGY
How Alberei structures the problem of humour generation. Register, structure, cultural frame.

USE CASES
Five product shapes worked through end to end, with integration patterns.

API REFERENCE
`alberei.de/api` · the complete request, response, and status code table.

Questions: `api@alberei.de`.